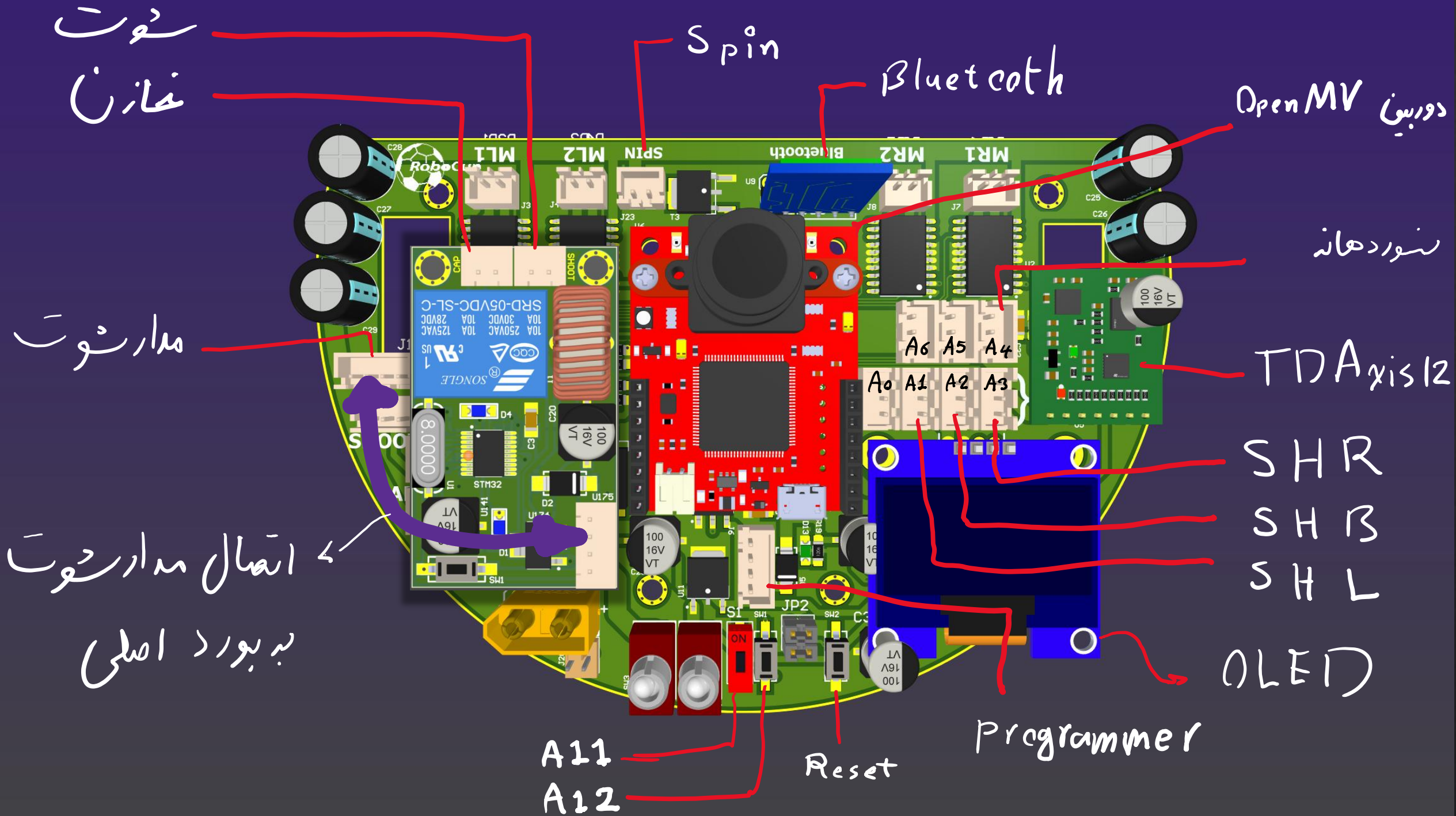


ربات فوتبالیست وزن آزاد

برنامه نویسی و راه اندازی ربات

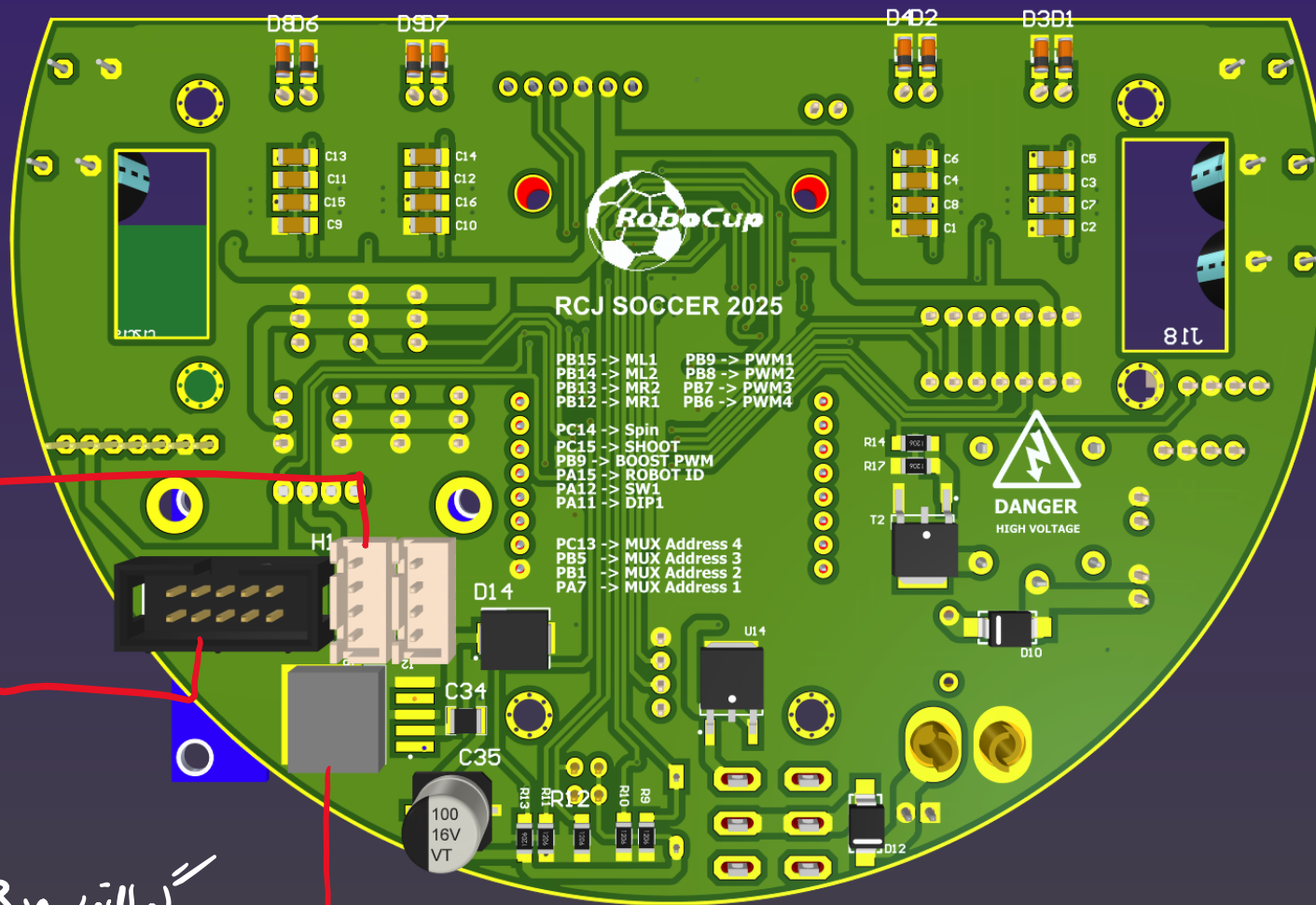
بهمن 1404
فرحان دائمی



سنورهای کف

کانکتور Dixy

رگولاتور 3.3v



مدار شوت

رله 5 ولت 10 آمپر

== > Cap+
== > Cap-
== > Solenoid-
== > Solenoid+

مانفست FDD6690

دیود M7

مانفست IRF540

درایور مانفست

رگولاتور 7805

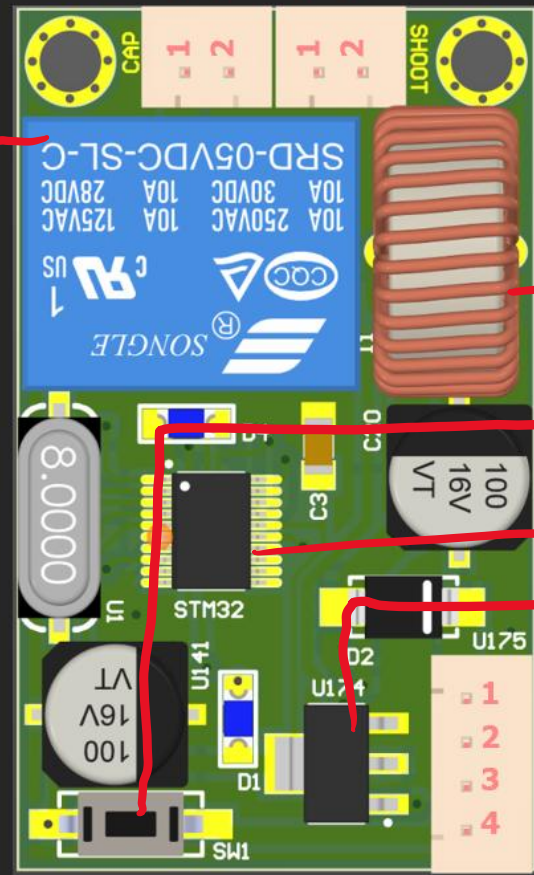
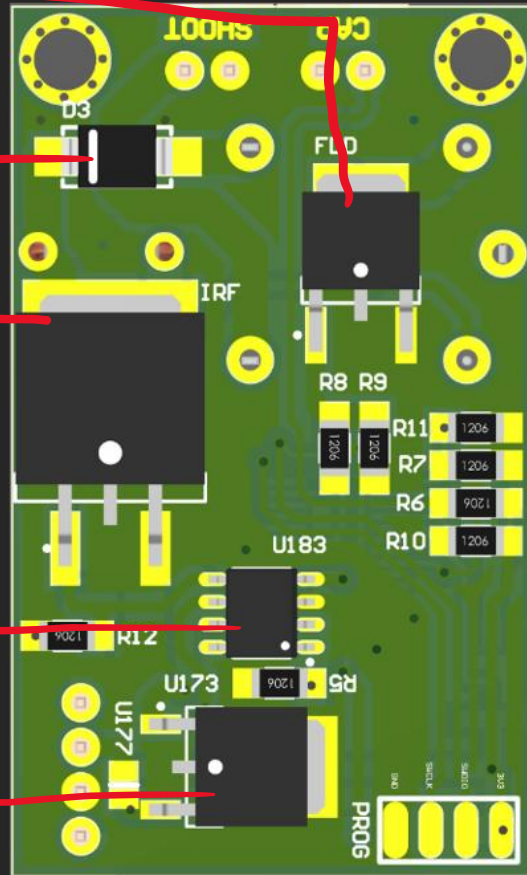
سلف 100 μ H

کلید شوت

STM32F030

رگولاتور AMS 1117

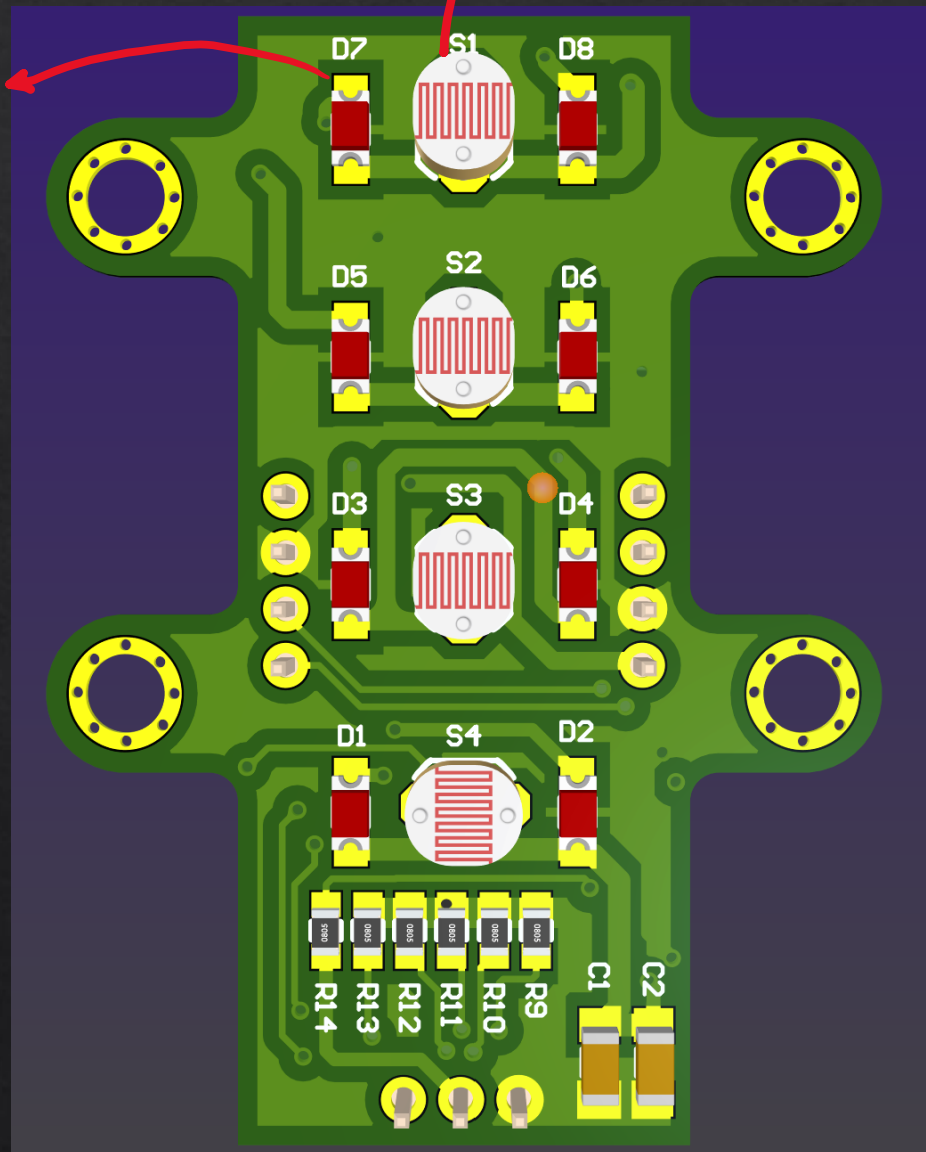
1 == > +12v
2 == > +5v (NC)
3 == > GND
4 == > Relay



ماژول سنسور LDR

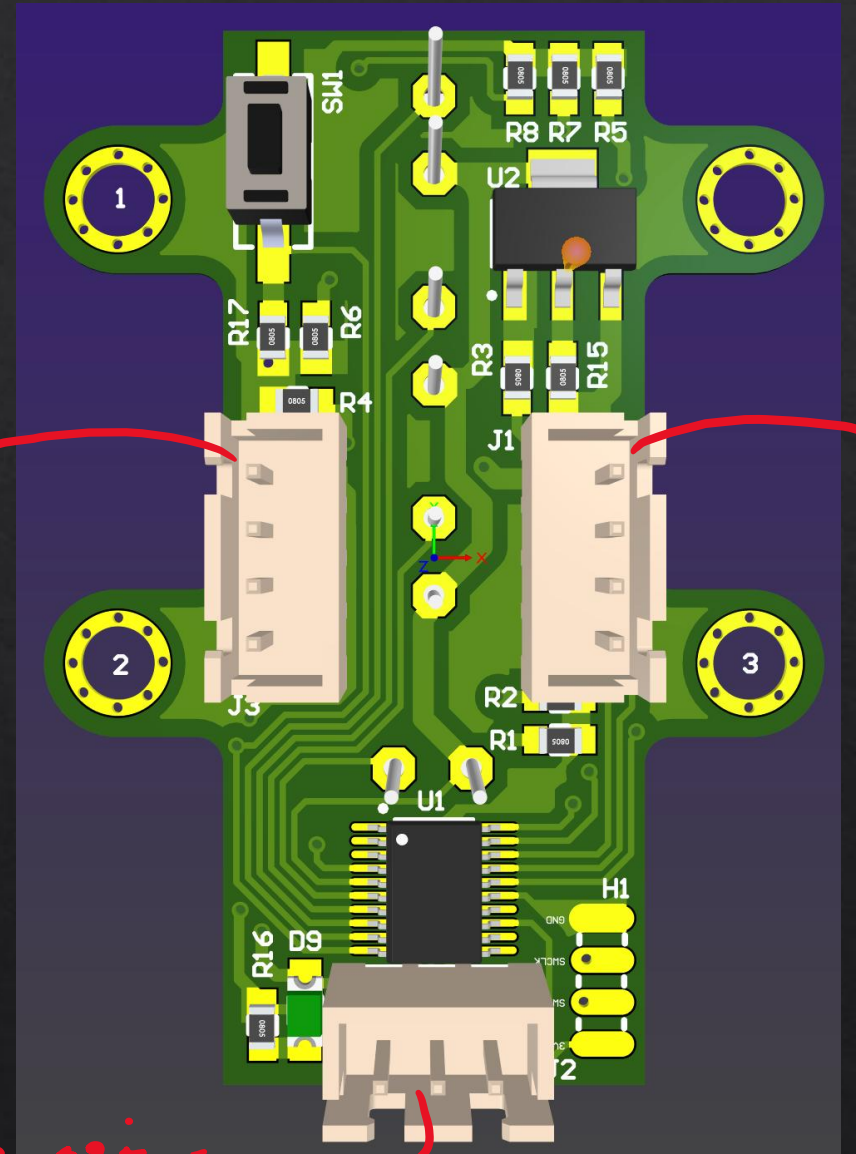
LDR

LED
white



پورت ۱۲C

پورت
۱۲C



پورت خروجی دیجیتال

تعاریف اولیه

```
#include <Wire.h>
#include <TDAxis12.h>
#include <Adafruit_SH1106_STM32.h>
#include <LDR.h>
#include <libmaple/iwdg.h>

#define MAX_SPEED 65000
#define HIGH_SPEED 65000
#define MID_SPEED 50000
#define LOW_SPEED 30000
#define VERYLOW_SPEED 20000
#define Team_Yellow true
#define SPIN_ON digitalWrite(PC14, 1)
#define SPIN_OFF digitalWrite(PC14, 0)
#define BALL_IN_KICKER digitalRead(PA4)
#define IS_Goaler !digitalRead(PA11)

Adafruit_SH1106 display(-1);
TwoWire i2c(2, I2C_FAST_MODE);
TDAxis12 gyro(&i2c, 0x10);
LDR ldr(&i2c);
```

```
bool OLED_EN = false;
bool use_gyro = true;
bool gyro_reverse = false;
bool already_shooted = false;
bool is_ball;
bool is_yellow;
bool is_blue;
bool is_goal;
bool arrived_to_goal = false;
float robot_angle;
float ball_x;
float ball_y;
float yellow_angle;
float yellow_x;
float yellow_y;
float blue_angle;
float blue_x;
float blue_y;
float goal_angle;
float goal_x;
float goal_y;
float ball_angle;
float ball_distance;
int robot_x_1 = 171; // OpenMV
int robot_y_1 = 118; // OpenMV
int robot_x_2 = 146; // OpenMV
int robot_y_2 = 134; // OpenMV
int robot_x;
int robot_y;
int robot_id;
int shl, shb, shr, D;
```


خواندن شماره ربات

```
void setup() {  
    if(digitalRead(PA12)) OLED_EN = true;  
    robot_id = digitalRead(PA15) + 1;  
    if (robot_id == 1) {  
        robot_x = robot_x_1;  
        robot_y = robot_y_1;  
    } else {  
        robot_x = robot_x_2;  
        robot_y = robot_y_2;  
    }  
}  
void loop() {  
  
}
```

آی دی ربات ها با یکی از پایه های میکروکنترلر به نام PA15 مشخص می شود. با استفاده از یک شرط می توان مشخص کرد که از کدام x و y استفاده شود.

از آنجایی که استفاده از OLED در مواقعی موجب هنگ کردن ربات می شود. می توان برنامه را طوری تنظیم کرد که اگر هنگام روشن شدن ربات کلید فشار داده شده بود از نمایشگر استفاده شود.

راه اندازی موتور ها

```
#define IN_1 PB15
#define IN_2 PB14
#define IN_3 PB13
#define IN_4 PB12
#define PWM_1 PB9
#define PWM_2 PB8
#define PWM_3 PB7
#define PWM_4 PB6
#define REVERSE_MOTORS true
```

یک فایل جدید باز کنید (ctrl+shift+n) و تعاریف مقابل را اضافه کنید. در اینجا هر موتور یک پایه IN برای کنترل جهت و یک پایه PWM برای کنترل سرعت دارد.

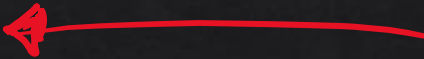
در صورتی جهت چرخش موتور ها برعکس باشند میتوان REVERSE_MOTORS را true قرار داد.

تابع تنظیمات موتورها

```
void init_motors() {  
    pinMode(IN_1, OUTPUT);  
    pinMode(IN_2, OUTPUT);  
    pinMode(IN_3, OUTPUT);  
    pinMode(IN_4, OUTPUT);  
    pinMode(PWM_1, PWM);  
    pinMode(PWM_2, PWM);  
    pinMode(PWM_3, PWM);  
    pinMode(PWM_4, PWM);  
    motor(0, 0, 0, 0);  
}
```

با استفاده از این تابع پایه های مربوط به موتور ها را تنظیم میکنیم. این تابع باید در تابع setup فراخوانی شود.

```
void setup() {  
    if(digitalRead(PA12)) OLED_EN = true;  
    robot_id = digitalRead(PA15) + 1;  
    if (robot_id == 1) {  
        robot_x = robot_x_1;  
        robot_y = robot_y_1;  
    } else {  
        robot_x = robot_x_2;  
        robot_y = robot_y_2;  
    }  
    init_motors();  
}
```



تابع موتور

```
void motor(int ML1, int ML2, int MR2, int MR1) {  
    if (ML1 > 0) {  
        digitalWrite(IN_1, 0);  
        pwmWrite(PWM_1, ML1);  
    } else {  
        digitalWrite(IN_1, 1);  
        pwmWrite(PWM_1, 65525 + ML1);  
    }  
    if (ML2 > 0) {  
        digitalWrite(IN_2, 0);  
        pwmWrite(PWM_2, ML2);  
    } else {  
        digitalWrite(IN_2, 1);  
        pwmWrite(PWM_2, 65525 + ML2);  
    }  
    if (MR2 > 0) {  
        digitalWrite(IN_3, 0);  
        pwmWrite(PWM_3, MR2);  
    } else {  
        digitalWrite(IN_3, 1);  
        pwmWrite(PWM_3, 65525 + MR2);  
    }  
    if (MR1 > 0) {  
        digitalWrite(IN_4, 0);  
        pwmWrite(PWM_4, MR1);  
    } else {  
        digitalWrite(IN_4, 1);  
        pwmWrite(PWM_4, 65525 + MR1);  
    }  
}
```

تابع موتور یک دستور برای مشخص کردن سرعت هریک از موتورهاست. در ادامه تغییراتی در این تابع خواهیم داد تا حرکت ربات بدون چرخش باشد و همیشه جهت آن به سمت جلو باشد.



جلوگیری از سرریز شدن pwm

```
void motor(int ML1, int ML2, int MR2, int MR1)
{
    if (REVERSE MOTORS) {
        ML1 = -ML1;
        ML2 = -ML2;
        MR2 = -MR2;
        MR1 = -MR1;
    }
```

برعکس شدن موتورها →

```
if (ML1 > MAX_SPEED) ML1 = MAX_SPEED;
if (ML1 < -MAX_SPEED) ML1 = -MAX_SPEED;
if (ML2 > MAX_SPEED) ML2 = MAX_SPEED;
if (ML2 < -MAX_SPEED) ML2 = -MAX_SPEED;
if (MR2 > MAX_SPEED) MR2 = MAX_SPEED;
if (MR2 < -MAX_SPEED) MR2 = -MAX_SPEED;
if (MR1 > MAX_SPEED) MR1 = MAX_SPEED;
if (MR1 < -MAX_SPEED) MR1 = -MAX_SPEED;
```

برنامه های قبلی

```
}
```

سرعت موتور با pwm مشخص می شود. pwm میکروکنترلرهای STM32 می تواند مقداری بین 0 تا 65535 داشته باشد. اگر مقدار pwm بیشتر از 65535 باشد مجدداً از صفر شروع می شود. این اتفاق overflow یا سرریز شدن رجیستر pwm گفته می شود.

به همین خاطر با استفاده از دو شرط برای هر موتور مقدار سرعت آن را به 65535- تا 65535+ محدود می کنیم.

آزمایش تابع موتور

حال با استفاده از تابع موتور به هر یک از موتور ها دستور می‌دهیم در جهت مثبت بچرخند اینکار باعث چرخش ربات در جهت عقربه های ساعت می‌شود.

تمرین: برنامه ای بنویسید که ربات 1 ثانیه جلو و 1 ثانیه عقب حرکت کند.

```
void loop() {  
    motor(LOW_SPEED, LOW_SPEED, LOW_SPEED, LOW_SPEED);  
}
```

تابع move

تابع move تابعی با دو متغیر ورودی است که به ما این قابلیت را می‌دهد تا به ربات دستور دهیم به سمت یک زاویه خاص با سرعت مشخصی حرکت کند. در برنامه زیر a زاویه و v سرعت می‌باشد. محاسبه سرعت موتورها با استفاده از توابع مثلثاتی انجام می‌شود.

```
void move(float a, int v) {  
    if (a > 360) a -= 360;  
    if (a < 0) a += 360;  
    int x = v * cos(a * M_PI / 180);  
    int y = v * sin(a * M_PI / 180);  
    motor((x + y), (x - y), (-x - y), (y - x));  
}
```

تابع moveXY

تابع moveXY یک تابع برای حرکت در راستای محورهای X و Y و چرخش با سرعت زاویه ای W است. در اینجا محور Y به سمت جلوی ربات و محور X به سمت راست ربات است. سرعت زاویه ای W هم در جهت عقربه های ساعت می باشد.

```
void moveXY(float Vx, float Vy, float w) {  
    int V11 = Vx + Vy + w;  
    int V12 = -Vx + Vy + w;  
    int Vr2 = -Vx - Vy + w;  
    int Vr1 = Vx - Vy + w;  
    motor(V11, V12, Vr2, Vr1);  
}
```


تابع moveXY

تابع moveXY یک تابع برای حرکت در راستای محورهای X و Y و چرخش با سرعت زاویه ای W است. در اینجا محور Y به سمت جلوی ربات و محور X به سمت راست ربات است. سرعت زاویه ای W هم در جهت عقربه های ساعت می باشد.

```
void moveXY(float Vx, float Vy, float w) {  
    int V11 = Vx + Vy + w;  
    int V12 = -Vx + Vy + w;  
    int Vr2 = -Vx - Vy + w;  
    int Vr1 = Vx - Vy + w;  
    motor(V11, V12, Vr2, Vr1);  
}
```

تابع stop

تابع stop جهت استادن ربات استفاده می‌گردد.

```
void stop() {  
    motor(0, 0, 0, 0);  
}
```

تست توابع حرکتی

```
void loop() {  
    for(int i=0; i<360; i=i+10){  
        move(i, LOW_SPEED);  
        delay(10);  
    }  
}
```

حال با استفاده از توابع نوشته شده می‌توان هر حرکت دلخواهی انجام داد. به عنوان مثال با برنامه روبه رو ربات حول یک نقطه می‌چرخد.

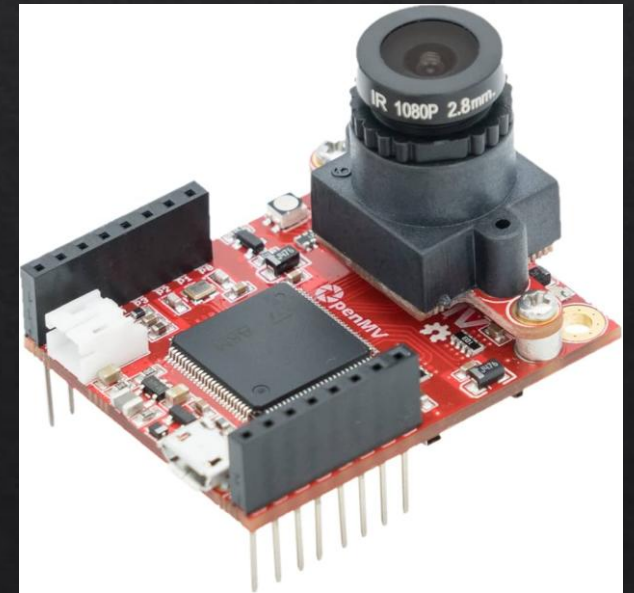
تمرین 1: برنامه ای بنویسید که ربات بر روی یک 8 ضلعی حرکت کند.

تمرین 2: برنامه ای بنویسید که ربات برایندی از حرکت جلو و چرخش به راست را انجام دهد

راه اندازی دوربین openMV

```
void setup() {  
    برنامه های قبلی  
    i2c.begin();  
}
```

ابتدا در تابع setup باید پروتوکل i2c را شروع کنیم.




```

void read_MV() {
    i2c.requestFrom(0x42, 12);
    if (i2c.available() == 12) {
        uint8_t buffer[12];
        for (int i = 0; i < 12; i++) {
            buffer[i] = i2c.read();
        }
        ball_x = buffer[0] | (buffer[1] << 8);
        ball_y = buffer[2] | (buffer[3] << 8);
        yellow_x = buffer[4] | (buffer[5] << 8);
        yellow_y = buffer[6] | (buffer[7] << 8);
        blue_x = buffer[8] | (buffer[9] << 8);
        blue_y = buffer[10] | (buffer[11] << 8);
        ball_angle = degrees(atan2(robot_x - ball_x, robot_y - ball_y));
        yellow_angle = degrees(atan2(robot_x - yellow_x, robot_y - yellow_y));
        blue_angle = degrees(atan2(robot_x - blue_x, robot_y - blue_y));
        ball_distance = sqrt(pow(robot_x - ball_x, 2) + pow(robot_y - ball_y, 2));
        if (ball_angle > 180) ball_angle -= 360;
        if (ball_angle < -180) ball_angle += 360;
        if (ball_x == 0 & ball_y == 0) is_ball = false;
        else is_ball = true;
        if (yellow_x == 0 && yellow_y == 0) is_yellow = false;
        else is_yellow = true;
        if (blue_x == 0 && blue_y == 0) is_blue = false;
        else is_blue = true;
    }
}

```

یک فایل جدید باز کرده و در آن تابع read_MV را تعریف کنید. در این تابع اطلاعات مربوط به دوربین که شامل مختصات توپ و دروازه ها می‌شود از طریق ارتباط i2C دریافت شده و زاویه توپ و فاصله ربات از آن با استفاده توابع ریاضی محاسبه می‌شود.

راه اندازی دوربین openMV

```
void loop() {  
    read_MV();  
}
```

در تابع loop تابع read_MV را فراخوانی کنید

راه اندازی نمایشگر OLED

```
int page = 0;
void oled_init(){
    if(!OLED_EN) return;
    display.begin(0x3c);
    display.setTextColor(WHITE);
    display.setTextSize(4);
    display.clearDisplay();
    display.setCursor(0, 10);
    display.print("");
    display.print(robot_id);
    display.display();
    delay(500);
    display.setTextSize(1);
    display.display();
}
```

یک فایل جدید باز کرده و در آن تابع روبه رو را تعریف کنید. این تابع تنظیمات مربوط به نمایشگر را انجام میدهد و با روشن شدن ربات، شماره ربات را روی آن به مدت 500 میلی ثانیه چاپ می‌کند. در تابع setup این تابع را فراخوانی کنید.

```
void setup() {
    برنامه های قبلی
    oled_init();
    i2c.begin();
}
```

```
void print_all(){
    if(!OLED_EN) return;
    if(digitalRead(PA12)){
        page++;
        page%=3;
        while(digitalRead(PA12));
    }
    display.clearDisplay();
    if(page == 0){
        display.setCursor(0, 20);
        display.print("Xb:");
        display.println(ball_x);
        display.print("Yb: ");
        display.println(ball_y);
        display.print("db: ");
        display.println(ball_distance);
        display.print("id: ");
        display.println(robot_id);
        display.println(IS_Goaler ? "Goaler" : "Forward");
    }
}
```

```
else if(page == 1){
    display.setCursor(0, 20);
    display.print("F:");
    display.println(ldr.front);
    display.print("R:");
    display.println(ldr.right);
    display.print("B:");
    display.println(ldr.back);
    display.print("L:");
    display.println(ldr.left);
}
else if(page == 2){
    display.setCursor(0, 20);
    display.print("shr:");
    display.println(shr);
    display.print("shb:");
    display.println(shb);
    display.print("shl:");
    display.println(shl);
}
display.display();
}
```


راه اندازی نمایشگر OLED

```
void loop() {  
    read_MV();  
    print_all();  
}
```

در تابع loop تابع بعد از read_MV تابع print_all را فراخوانی کرده تا اطلاعات روی نمایشگر نمایش داده شود. توجه داشته باشید که بعد از آپلود برنامه روی ربات باید ربات را خاموش کرده، کلید PA11 را فشار داده و نگه دارید سپس ربات را روشن کنید تا oled فعال شود.

با این کار تنها زمانی که نیاز به نمایش اطلاعات داریم oled فعال می‌شود و در حالت عادی خاموش می‌ماند و از هنگ کردن ربات جلوگیری میکند.

نوشتن یک تعقیب توپ ساده

```
void loop() {  
    read_MV();  
    print_all();  
    if (is_ball) {  
        move(ball_angle, MID_SPEED);  
    }  
    else{  
        stop();  
    }  
}
```

برای تعقیب توپ ساده کافی است با یک شرط بنویسیم اگر توپ دیده شد حرکت به سمت زاویه توپ انجام شود. اینکار باعث می شود ربات بدون چرخش به طرف توپ حرکت کند. در مرحله بعد با شیفِت زاویه ای به ربات می گوییم به پشت توپ برود تا توپ در دهانه ربات قرار گیرد.

نوشتن شیفت حرکت ها

در این برنامه شیفت حرکت ها با ضریب 1.5 انجام می شود. هر چقدر این ضریب بیشتر باشد ربات فاصله بیشتری از توپ می گیرد و شیفت بیشتری می دهد. تابع clamp حداکثر و حداقل مقدار شیفت زاویه را مشخص می کند که در اسلاید بعد توضیح داده شده.

```
void loop() {  
    read_MV();  
    print_all();  
    if (is_ball) {  
        float shift = clamp(ball_angle * 1.5, -60, 60);  
        move(ball_angle + shift, MID_SPEED);  
    } else {  
        stop();  
    }  
}
```

تعریف تابع clamp

تابع clamp زمانی استفاده می‌شود که نمی‌خواهیم یک مقدار از یک حداکثر و حداقل بزرگتر یا کوچکتر نشود. به عنوان مثال در تابع موتور که سرعت از 65535 نباید بیشتر می‌شد می‌توانستیم از این تابع استفاده کنیم. این تابع را در فایل motor تعریف کنید.

```
float clamp(float val, float minn, float maxx) {  
    if (val > maxx) return maxx;  
    if (val < minn) return minn;  
    return val;  
}
```


راه اندازی سنسور TDAxis12

برای گرفتن زاویه ربات کافی است دستور خواندن
ژیروسکوپ را به ابتدای حلقه loop اضافه کنیم:

```
void loop() {  
    robot_angle = gyro.read();  
    read_MV();  
    print_all();  
    برنامه های قبلی  
}
```

```
void motor(int ML1, int ML2, int MR2, int MR1) {  
    if (use_gyro) {  
        float correction = robot_angle;  
        if (gyro_reverse) {  
            if (correction >= 0) correction -= 180;  
            else correction += 180;  
        }  
        correction = clamp(correction, -90, 90);  
        ML1 += correction * 400;  
        ML2 += correction * 400;  
        MR2 += correction * 400;  
        MR1 += correction * 400;  
    }  
    برنامه های قبلی  
}
```

در تابع موتور دستورات روبه رو را اضافه کنید تا ربات در جهت دروازه حریف خودش را صاف کند و در صورت نیاز در با متغیر gyro_reverse ربات را به سمت دروازه خودی بچرخانیم.

راه اندازی شوت و اسپین

```
void shoot_init() {  
    pinMode(PA4, INPUT_PULLUP);  
    pinMode(PC14, OUTPUT);  
    pinMode(PC15, OUTPUT);  
}  
  
void shoot() {  
    for(int i=0; i<2; i++){  
        update_all();  
        move_inside();  
        digitalWrite(PC15, 1);  
    }  
    digitalWrite(PC15, 0);  
}
```

یک فایل جدید باز کرده و در آن دو تابع روبه رو را بنویسید. تابع shoot_init را در setup فراخوانی کنید تا تنظیمات اولیه مربوط به شوت و اسپین انجام گردد.

```
void setup() {  
    برنامه های قبلی  
    shoot_init();  
    init_motors();  
    i2c.begin();  
    oled_init();  
}
```

استفاده از شوت

```
void loop() {  
  robot_angle = gyro.read();  
  read_MV();  
  print_all();  
  if (BALL_IN_KICKER) {  
    shoot();  
  } else if (is_ball) {  
    float shift = clamp(ball_angle * 1.5, -60, 60);  
    move(ball_angle + shift, MID_SPEED);  
  } else {  
    stop();  
  }  
}
```

با استفاده از یک شرط می‌گوییم اگر توپ در دهانه ربات قرار گرفت شوت بزند.

استفاده از اسپین

اسپین زمانی باید روشن شود که توپ در دهانه ربات قرار گرفته یا زاویه توپ جلوی ربات باشد و توپ توسط دوربین دیده شود. در غیر این صورت باید خاموش شود.

```
void loop() {  
    if ((is_ball && ball_angle < 20 && ball_angle > -20) || BALL_IN_KICKER) SPIN_ON;  
    else SPIN_OFF;  
    برنامه های قبلی  
}
```

خواندن سنسورهای کف

```
void loop() {  
    ldr.read();  
    برنامه های قبلی  
}
```

خواندن سنسورهای کف در تابع loop انجام می شود.

تعریف تابع moveSec در فایل موتور:

```
void moveSec(float a, int v, float sec) {  
    for (int i = 0; i < sec; i++) {  
        if(move_inside()) move(a, v);  
        update_all();  
    }  
}
```

الگوریتم اوت

یک فایل جدید ساخته و تابع اوت را به صورت زیر تعریف کنید.

```
#define LDR_SENS 0
#define MOVE_INSIDE_SEC 6
bool checking_out = false;
void out() {
    checking_out = true;
    if (ldr.front > LDR_SENS && ldr.right > LDR_SENS) moveSec(225, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.front > LDR_SENS && ldr.left > LDR_SENS) moveSec(135, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.back > LDR_SENS && ldr.right > LDR_SENS) moveSec(315, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.back > LDR_SENS && ldr.left > LDR_SENS) moveSec(45, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.front > LDR_SENS) moveSec(180, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.right > LDR_SENS) moveSec(270, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.back > LDR_SENS) moveSec(0, HIGH_SPEED, MOVE_INSIDE_SEC);
    else if (ldr.left > LDR_SENS) moveSec(90, HIGH_SPEED, MOVE_INSIDE_SEC);
    else {
        checking_out = false;
        return;
    }
    float out_ball_angle = ball_angle;
    while (is_ball && abs(out_ball_angle - ball_angle) < 30) {
        update_all();
        if (move_inside()) stop();
    }
    checking_out = false;
}
```

الگوریتم اوت

تعریف تابع move_inside این تابع با بررسی سنسور های کف حرکت مخالف را انجام میدهد تا از ربات از خط اوت دور شود.

```
bool move_inside() {  
    if (ldr.front > LDR_SENS && ldr.right > LDR_SENS) move(225, HIGH_SPEED);  
    else if (ldr.front > LDR_SENS && ldr.left > LDR_SENS) move(135, HIGH_SPEED);  
    else if (ldr.back > LDR_SENS && ldr.right > LDR_SENS) move(315, HIGH_SPEED);  
    else if (ldr.back > LDR_SENS && ldr.left > LDR_SENS) move(45, HIGH_SPEED);  
    else if (ldr.front > LDR_SENS) move(180, HIGH_SPEED);  
    else if (ldr.right > LDR_SENS) move(270, HIGH_SPEED);  
    else if (ldr.back > LDR_SENS) move(0, HIGH_SPEED);  
    else if (ldr.left > LDR_SENS) move(90, HIGH_SPEED);  
    else return true;  
    return false;  
}
```


الگوریتم اوت

فراخوانی تابع اوت در loop:

```
void loop() {  
    ldr.read();  
    robot_angle = gyro.read();  
    read_MV();  
    print_all();  
    out();  
    برنامه های قبلی  
}
```

با آپلود این برنامه ربات نباید از خط اوت خارج شود. اما ایراداتی در پردازش سنسورهای کف وجود دارد که باعث می شود ربات در برخی مواقع از خط اوت گذر کند. در ادامه به حل این مشکل خواهیم پرداخت.

ساخت تابع آپدیت

یک فایل جدید به نام AI ساخته و تابع زیر را در آن تعریف کنید. این تابع تمامی سنسورها را آپدیت می‌کند. و در صورتی که سنسورهای کف خط اوت را تشخیص داده باشند از خواندن سنسورهای دیگر صرف نظر می‌کند. اینکار باعث می‌شود ربات سریع تر تابع Out را اجرا کند و از خط فاصله بگیرد.

```
void update_all() {  
    ldr.read();  
    if (!out_detected()) {  
        robot_angle = gyro.read();  
        read_MV();  
        print_all();  
    }  
    if ((is_ball && ball_angle < 20 && ball_angle > -20) || BALL_IN_KICKER) SPIN_ON;  
    else SPIN_OFF;  
    if (!BALL_IN_KICKER) already_shooted = false;  
}
```

استفاده از تابع update در loop

در loop به جای خواندن سنسورها در چند خط می‌توانید در یک خط از تابع update_all برای خواندن همه سنسورها استفاده کنید.

```
void loop() {  
  update_all();  
  if (BALL_IN_KICKER) {  
    shoot();  
  } else if (is_ball) {  
    float shift = clamp(ball_angle * 1.5, -60, 60);  
    move(ball_angle + shift, MID_SPEED);  
  } else {  
    stop();  
  }  
}
```

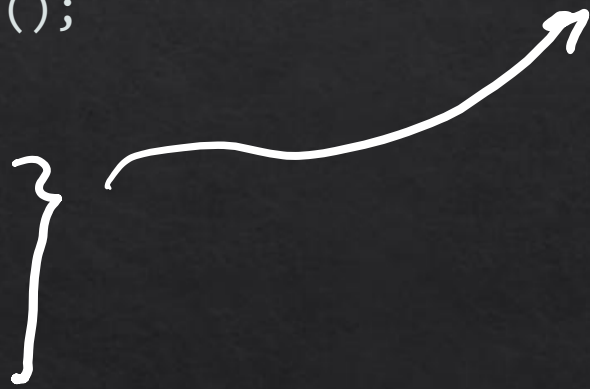
تعریف تابع out_detected

این تابع خیلی سریع سنسورهای کف را خوانده و اگر سنسوری خط اوت را تشخیص دهد مقدار true را برمیگرداند. همچنین به خاطر وجود متغیر checking_out اگر در حال بررسی اوت باشیم این تابع false را برمیگرداند تا در یک لوپ بی نهایت گیر نیفتد. (چون update_all در moveSec فراخوانی می‌شود و در تابع out وقتی moveSec انجام می‌دهیم سنسورها باید آپدیت شوند)

```
bool out_detected() {  
    if(ldr.back > LDR_SENS) arrived_to_goal = true;  
    if(checking_out || IS_Goaler) return false;  
    if (ldr.front > LDR_SENS || ldr.right > LDR_SENS || ldr.back > LDR_SENS ||  
        ldr.left > LDR_SENS) return true;  
    return false;  
}
```


بازگشت به دروازه

```
void update_all() {  
    ldr.read();  
    if (!out_detected()) {  
        robot_angle = gyro.read();  
        read_MV();  
        print_all();  
        shr = analogRead(PA3);  
        shb = analogRead(PA2);  
        shl = analogRead(PA1);  
        D = (shr - shl) * 20;  
    }  
    if ((is_ball && ball_angle < 20 && ball_angle > -20) || BALL_IN_KICKER) SPIN_ON;  
    else SPIN_OFF;  
    if (!BALL_IN_KICKER) already_shooted = false;  
    iwdg_feed(); // reload watchdog  
}
```



در ابتدا باید مقدار سنسورهای فاصله سنج
لیزری در تابع update_all خوانده شود.

بازگشت به دروازه

مانند برنامه زیر در آخرین else دستور moveXY را نوشته و D (اختلاف فاصله چپ و راست) را به عنوان سرعت در راستای x (چپ و راست ربات) و (800 - shb) با یک ضریب مشخص به عنوان سرعت در راستای محور y (جلو و عقب) بدهید. اینکار باعث می‌شود ربات به جلوی دروازه خودی برود. ضریب 80 هر چه بیشتر شود سرعت حرکت ربات به عقب بیشتر می‌شود. 800 هم فاصله ای است که ربات از پشت خود حفظ می‌کند.

```
void loop() {  
    update_all();  
    if (BALL_IN_KICKER) {  
        shoot();  
    } else if (is_ball) {  
        float shift = clamp(ball_angle * 1.5, -60, 60);  
        move(ball_angle + shift, MID_SPEED);  
    } else {  
        moveXY(D, (800 - shb) * 80, 0);  
    }  
}
```

چرخاندن توپ به سمت عقب

این الگوریتم زمانی کاربرد دارد که ربات ما با ربات حریف درگیر می‌شود و بازی در یک نقطه ثابت می‌ماند. با استفاده از قابلیت اسپین می‌توان در چنین مواقعی توپ را از دید ربات های حریف پنهان کرد. اما باید توجه داشته باشیم که این الگوریتم تنها زمانی باید اجرا شود که ربات با یک بار شوت زدن موفق به خروج توپ از دهانه خود نشود. به همین خاطر متغیری به نام `already_shooted` تعریف شده که هنگامی که توپ وارد دهانه می‌شود با یک بار شوت زدن این متغیر `true` می‌شود و اگر توپ از دهانه ربات خارج شود این متغیر `false` می‌شود.

در تابع `loop` تنها دو تابع `update_all` و `forward` را تعریف کنید تا در ادامه به جزئیات تابع `forward` بپردازیم.

```
void loop() {  
    update_all();  
    forward();  
}
```

چرخاندن توپ به سمت عقب

```
void forward() {  
    out();  
    if (BALL_IN_KICKER) {  
        goal();  
    } else if (is_ball) {  
        catch_ball();  
    } else {  
        moveXY(D, (800 - shb) * 80, 0);  
    }  
}
```

تابع forward سه وضعیت کلی را مورد بررسی قرار می‌دهد و برای هر حالت یک تابع را صدا می‌زند.

- توپ داخل دهانه باشد
- توپ دیده شود
- توپ دیده نشود و داخل دهانه هم نباشد

تابع catch_ball

این تابع همان شیفت حرکتی است که پیش از این مورد استفاده قرار می‌گرفت. از آنجایی که قصد چرخش ربات را در ادامه داریم باید در تمام حالت‌ها وضعیت استفاده از سنسور زاویه را مشخص کنیم.

```
void catch_ball() {  
    gyro_reverse = false;  
    use_gyro = true;  
    float shift = clamp(ball_angle * 1.5, -60, 60);  
    move(ball_angle + shift, MID_SPEED);  
}
```

```

int goal_time_out = 0;
void goal() {
    if (already_shooted) {
        if (arrived_to_goal) {
            gyro_reverse = false;
            use_gyro = true;
            move(0, VERYLOW_SPEED);
            if (robot_angle > -20 && robot_angle < 20 && goal_time_out > 80) already_shooted = false;
            goal_time_out++;
        } else {
            gyro_reverse = true;
            use_gyro = true;
            move(180, LOW_SPEED);
            goal_time_out = 0;
        }
    } else {
        gyro_reverse = false;
        use_gyro = true;
        already_shooted = true;
        arrived_to_goal = false;
        move(0, HIGH_SPEED);
        shoot();
    }
}

```

تابع goal

این تابع دو بخش اصلی دارد. اگر already_shooted برابر true باشد یعنی قبلاً شوت زده شده و حالا ربات باید بچرخد و به عقب حرکت کند تا به دروازه حریف برسد. مقدار 80 که برای goal_time_out تنظیم شده مدت زمانی است که ربات صبر میکند تا خازن شوت آن شارژ شده و سپس شوت بزند. اگرهم already_shooted برابر false باشد یک بار شوت میزند و آن را true میکند تا در ادامه در صورت خارج نشدن توپ ربات بچرخد.

الگوریتم دروازه بان

هر ربات با استفاده از کلید deep switch که بر پایه PA11 قرار گرفته می‌تواند روی حالت دروازه بان یا مهاجم تنظیم شود. برای اضافه کردن برنامه دروازه بان حلقه loop را به این صورت تغییر دهید:

```
void loop() {  
    update_all();  
    if (IS_Goaler) defend_goal();  
    else forward();  
}
```

الگوریتم دروازه بان

```
void defend_goal() {  
    if (BALL_IN_KICKER) {  
        goal();  
    } else if (is_ball) {  
        if (ball_distance < 70 || ball_angle > 120 || ball_angle < -120) {  
            out();  
            catch_ball();  
        } else {  
            int Vx = (robot_x - ball_x) * 2000;  
            int Vy = (800 - shb) * 80;  
            if (shl < 400 && Vx < 0) Vx = 0;  
            if (shr < 400 && Vx > 0) Vx = 0;  
            moveXY(Vx, Vy, 0);  
        }  
    } else {  
        come_back();  
    }  
}
```

تابع defend_goal را به این صورت در AI تعریف کنید. این تابع در صورت نزدیک بودن توپ همان کارهای ربات مهاجم را انجام می‌دهد. اما در صورت دور بودن توپ در راستای آن به چپ و راست حرکت میکند. عدد 800 فاصله ای است که ربات از پشت خود حفظ میکند. و ضریب 20000 سرعت چپ و راست شدن ربات را تعیین میکند. عدد 80 هم سرعت جلو و عقب رفتن ربات در هنگام دیده شدن توپ.